SmartQueue: A Self-Adaptive Multi-Counter Queue Management System following MAPE-K Loop

1st Canyang (Richard) Wang
University of Waterloo
Waterloo, Canada
c665wang@uwaterloo.ca, ID: 20882741
Group Number 9

2nd Shilin (Stanley) Zhang
University of Waterloo
Waterloo, Canada
s729zhan@uwaterloo.ca, ID: 20890775
Group Number 9

***Abstract*—Modern services are limited in resources and there are increasingly more people that requires services so wait time is a crucial aspects that services need to consider because if people needs to wait for a really long time for a service, customers of the service may not be happy. So the challenge is that how can we process all jobs so that no jobs are delayed beyond a fixed wait time. Traditional solution is to have a queue system that has a fixed scheduling policies and a fixed number of counters to serve customers. The emphasis is put on the scheduling policies where experts spend times to design a scheduling policies but there is no scheduling policies that works the best in all circumstances. The workload, arrival rate and processing time of jobs are unpredictable so it is really hard to design such scheduling policies and most services try and predicts the workload, arrival rate and processing time at a time and manually adjust scheduling policies and number of counters. But this requires lots of work by human so it is costly and it is really slow because experts need to monitor and analyze before they make any adjustment. So our project proposed a solution called SmartQueue which is a self-adaptive queueing system that uses the MAPE-K Loop structure and it automatically adjusts number of counters and scheduling policies when the environment changes. It dynamically adjusts number of counters and picks scheduling policies from either First in First out or Shortest Processing Times first based on real time metrics collected from a simulated environment driven by JMeter. SmartQueue continuously monitor metrics such as queue length and wait time and analyze the utilization of counters and heavy-tailed indicator to plan and execute adaptation actions to adjust system configuration. It is evaluated under three different workloads against a static queueing system with fixed scheduling policies (FIFO) and number of counters. Results shows that SmartQueue reduces the average wait time, queue length and number of jobs that has wait time beyond a specific threshold in all three workloads. This finding confirms the fact that an adaptive queueing system SmartQueue performs better than the traditional queueing system which has static number of counters and scheduling policies under an environment with unpredictable arrival rate and processing time.**

I. Introduction

Because modern systems, electronic technologies, and websites not only focus on correctness but also efficiency of completing a task, it is crucially important to manage resources and make changes so that the processing time is as fast as possible. As more and more people use these technologies and online platforms, it is almost impossible to have enough resources that accounts for all users without having users to wait so wait time is a key measurement to reduce in lots of these platforms and technologies. When wait time is considered, the first thing that comes to mind is a queueing system that decides which user gets the next available resource and traditional queueing designs relies on predicting the traffic intensity and processing time to fix a number of counters and queueing methods that process users request. However, this is rarely true in practices because in reality, the traffic intensity and processing time is unpredictable because there are lots of different factors that can impact these measurements. Another field of study related to queueing system is queueing theory which focus on designing a queueing methods that leads to minimum wait time but there is no queueing methods that leads to minimum wait time in all circumstances. As a result of these unpredictable measurements, a static queueing system that has a fixed number of counters and queueing methods could lead to undesirable performance such as high wait time, underutilized resources, and poor quality of services. This issue is not limited to the technology industry, queueing up in line is everywhere in the real world from supermarket to call centers and even when people go to gas station to fill up

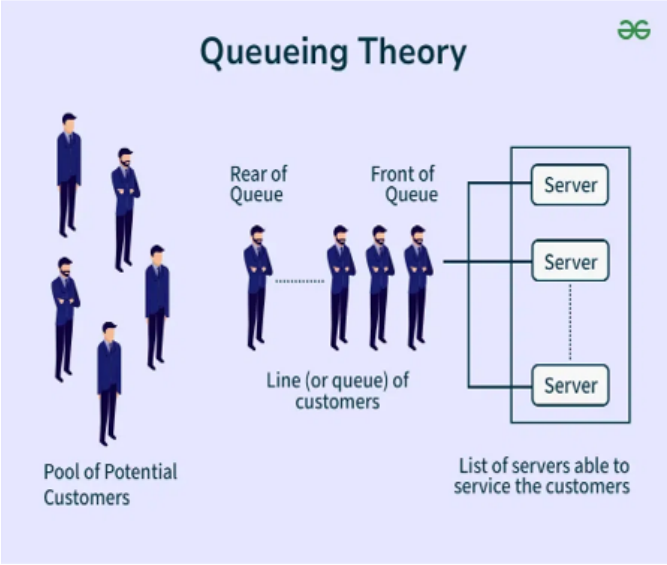

Fig. 1. This is a queue scenario that our project work in where there is a queue of customers and there are multiple server that serves customers.

their gas tank, they will need to wait in a queue. The traffic of a shop is often used to determine how popular the shop is but often shops with high traffic need to wait for a longer time which might lead to worse quality of service and lots of shops realized that a static queueing system may lead to poor quality of service so they manually adjust the number of counters. However, this requires lots of work from human and it requires expert analysis on when and how much to add or remove a counter which is really costly and slow.

So this project proposed a self-adaptive solution that utilizes MAPE-K Loop to address these challenges in the context of a multi-counter queueing environment and explore how having dynamic number of counters and queueing methods help to reduce wait time and increase quality of services. It makes both the queue and the counters adaptive by allowing the queue to change its queueing methods between First in First Out (FIFO) and Shortest Processing Time First (SPT) and the number of counters to increase and decrease so that the queueing system that the project proposed can adapt to different circumstances under unpredictable traffic.

II. Methodology

Before talking about the actual design of our project, we first need to initialize some quality requirements so we know what we want to achieve and how we can evaluate our solution on the effectiveness. We have organized the quality requirements of our project into a table named **TABLE I**.

TABLE I
Quality Requirements

Quality Requirements
Service Quality: Jobs with wait time above 5 seconds are considered as failed jobs
Effectiveness: Percentage of failed jobs should be reduced by at least 30% compared to static configuration
Resource Constraints: Use at most 10% more resources compared to static configuration
Stability: Make at most 1 adaptation every 5 seconds

The service quality is what we used to evaluate the performance of each jobs where we consider jobs that waits for more than 5 seconds as waiting too long so consider them as failed. In the proposal that we previously submitted, we used 1 seconds instead of 5 seconds but when we actually try to run it with 1 seconds, almost all jobs failed which is really not useful for evaluation. We think this is because JMeter sends all jobs in a short amount of period and our simulation is not able to process all the jobs that JMeter sends so jobs stack up in the queue really quickly causing all the jobs to fail so we decided to loose up the requirements on failed jobs to 5 seconds for better evaluation. The effectiveness and constraints quality requirements are used to evaluate the performance of our self-adaptive queueing system where we want to use at most 10% more resources to achieve a 30% decrease in the number of failed jobs. The reason why we want to see a 30% decrease in the percentage is because we want the improvement to be significant and actually worth the cost of making adaptation actions instead of a static configuration and the resource constraint is to make sure that we do not use much more resources to achieve the better performance. If we use 30% more resources to achieve a 30% improvement, there is really no meaning to the improvement because by using more resources, we should see similar improvements. The last requirement is our consideration on cost of doing adaptation actions so we allow only 1 adaptation per 5 seconds to make sure the system is stable.

To try to achieve less failed jobs, we come up with a list of adaptation goals to accomplish these requirements (**TABLE II**).

TABLE II
Adaptation Goals and Actions

Adaptation Goals	Adaptation Action
Keep utilization of counters between 0.4 to 0.8	Increase or Decrease number of counters considering wait time
Keep wait time below 5 seconds	Increase number of counters or switch queueing methods

These two primary adaptation goals are first we want to keep good utilization of the number of counters so if the utilization of counters is too low and the wait time is reasonable, we should decrease number of counters and if the utilization is too high and the wait time is also high, we increase. The second adaptation goals corresponds to reducing wait time where if we have a fairly high wait time, then should consider increasing the number of counters or change the queueing methods if needed.

After specifying goals of our solution, we then start to implement the actual solution and simulation of our project. The first part that we consider is the simulation part of the project where we need to make sure the simulation is feasible and whether we want to use a database to run the simulation or we generate our own data to run the simulation. We tried searching for database for our simulation but we found that most database have timestamp that is different from current time and a lot of them have processing time that is really long compare to our threshold of 5 seconds so we decided to use JMeter to generate requests and jobs for us. Another reason why we chose JMeter instead of existing database is because we want to keep our simulation and evaluation small and quick so existing database may not give us the degree of freedom compare to having JMeter generates requests for us randomly and we still have control over the maximum processing time.

After we decided that we want to use JMeter instead of existing database, we start to implement another part of the simulation which is the queueing system itself or the managing system in the context of self-adaptive system. We simply build this by constructing a queue that contains all jobs send by JMeter and construct multiple counters that handles each job and the counter will take the processing time that JMeter assign to each job to complete the job. This simulates the multi-counter queue system in real world and even though it is simple, it is enough for the purpose of our evaluation because it provides the necessary functionality that we need to run the simulation.

Next, we considered and designed the core of our project which is the MAPE-K loop that is used to manage the simple queue system. We will discuss the loop in four different parts following the structure of the loop which are Monitoring, Analyzing, Planning and Executing components [3].

The **Monitoring component** is used to collect and compute data and measurements that is needed to analyze the current state of the queueing system. We decided that the monitoring components collects data every 0.1 second. We make this decision because the maximum processing time of any job is 1 second so measurements change really quickly and we want to have enough measurements to analyze within the lifetime of a longest job. Having more measurements about jobs allow us to make more accurate analysis and planning and we think 10 measurements for longest possible jobs is enough to make an accurate decision. Another reason why we decided to monitor every 0.1 second is because if the system is overloaded with jobs, this could happen in a few hundreds of milliseconds so 0.1 second allows the MAPE-K Loop to detect this and act on it. We didn't choose a smaller number because we don't want our system to be loaded with data and having too much data can make our analysis focus on outliers meaning it is not stable. We wouldn't need to worry about too frequent adaptation because we have a hard restriction that we make one adaptation action every 5 seconds.

Then, we went on and discuss what are some measurements and data that the monitoring component should collect and compute for our analyzing component to do effective analysis. We decided to collect average, minimum and maximum wait time of the queue which gives us an idea of how loaded is the queue and the average service time among finished jobs to give us an idea of the processing time of completed jobs. On top of the average service time, it also collects the service time of each of the completed jobs so the analyzing components can evaluate whether the jobs in the queue is heavy-tailed or not. We also collect metrics like number of failed jobs based on our definition of failed jobs, the current length of the queue and an the number of jobs that arrived in the past seconds. The number of failed jobs is used primarily for evaluation only and the length of the queue is used to see how the length of the queue change as jobs finishes and arrives. The number of jobs that arrive in the past second is used to compute the estimated arrival rate which is the responsibility of analyzing components. The last metrics that the monitoring component collects is the number of jobs that counters currently process. It counts the number of jobs that has started but not finished and it is used to calculate the utilization of counters in the system.

After monitoring component finished collecting these metrics, the **analyzing component** uses these metrics to analyze the current state of the system. First, it calculates the estimated arrival rates using the formula $\hat{\lambda_t} = \alpha \times \lambda_{observe} + (1-\alpha) \times \hat{\lambda_{t-1}}$ where $\hat{\lambda_t}$ is the arrival rate at time t, $\lambda_{observe}$ is the actual arrival rate in the past second and α is a smooth constant which is typically 0.2 [2]. However, we realized that this is not really feasible with simulation style because in reality, the queue system never stop after a certain duration so this formula is appropriate. However, in our simulation context, JMeter sends all jobs within the duration and if we are not able to process the jobs on time, it will stack in the queue and our MAPE-K loop need to run much longer than the duration that the user give. In this case, the actual arrival rate in the past second is actually 0 because JMeter stopped sending requests which makes this arrival rate approaching 0 so under our context, this formula may not be appropriate. Originally, we plan to use the estimated arrival rate calculate a utilization score for the system to characterize how utilized are we with the counters but since the arrival rate is not accurate in this particular circumstances, we need an alternative for utilization function. Instead of using complex utilization function, we decided to use a simple and intuitive formula for the utilization of counters in the system which is $\frac{processing\ jobs}{number\ of\ counters}$. Basically, this formula evaluates what percentage of counters are currently processing jobs. The numerator is the number of jobs that are currently being processed by a counter but haven't finished so this percentage gives us an idea of how busy each counter is. We can make decisions like add or remove counters using this percentage. Even though the estimated arrival rate is not appropriate in our context, we still decide to keep it for illustration purposes so the user can see what the estimated arrival rate is. On top of number of counters, we can also change the queueing methods of the system and our project uses two queueing methods which are First in First out and Shortest processing times first. First in first out is just the order in which jobs comes in and shortest processing times first means we process jobs with shortest processing times first. Shortest processing times first has an advantage when the queue has multiple short jobs that has been delayed and waiting for few extremely long jobs so this is a property that we want to characterize in order to decide whether we should change queueing methods [1]. We calculate a number called heavy-tail indicator where we first calculate the median and 90th percentile of service time of completed jobs and if the fraction $\frac{median}{p90} > 1.4$, then we consider the queue as heavy-tailed so the indicator is 1. Otherwise, the indicator is 0. Basically, the fraction is saying that the top 10% of the jobs has at least 40% more processing time compared to the median which meets our description of when shortest processing times first has an advantage. We use 40% because we think anything less than 40% is not really significantly longer and because we restrict the processing time of jobs to be less than 1 seconds, any percentage less than 40% is even more insignificant. Another aspects that we discussed is that because we have processing times from JMeter, we don't want just use this time to determine whether it is heavy-tail or not. The reason why we decided to use completed jobs is because the processing times JMeter assigned to jobs is just the time required to process the jobs. It is not equivalent to service time that we observed after the job finishes. The actual and true service time may be delayed by other factors such as scheduling delays so we decided to use service time of completed jobs to calculate this indicator.

After analyzing component finishes computing the utiliza-

tion of counters and heavy-tail indicator, the **planning components** uses these numbers to come up with an adaptation plan. We have four different actions that could be performed which are add a counter, remove a counter, change the scheduling policy to First in First out or shortest processing time first. Consequently, we have four different conditions for these four actions. We define that if the counter utilization is greater than 0.8 and the average wait time is greater than one second, then we add a counter. In plain English, we add a counter if the utilization of counters is high and the wait time is fairly long. We choose one second as long wait time instead of five seconds which is our definition of failed jobs because we don't want to wait until it has failed jobs then add more counters. We want to be proactive so before any jobs fail, we add counters when the average wait time is a bit high. If the utilization of counters is less than 0.4 and the average wait time is less than 0.5 second, then we decrease a counter. This corresponds to the case of underutilized counters where if not all counters are utilized and the average wait time is fairly low, then we can safely remove a counter without impacting the quality of service. The last two actions are just switching between the two scheduling policy and as we discussed above, shortest processing time first is better when the jobs are heavy-tailed so when heavy-tail indicator is 1, we switch to shortest processing time first and otherwise, we switch to first in first out. Note that adding or removing counters and switching scheduling policy can happen in the same adaptation action. This means that we can add a counter and switch scheduling policy in one adaptation action. Note that the 5 seconds cooldown is also enforced in planning component where if the previously adaptation action is within 5 seconds from now, we do not make any adaptation actions. The reason why we put it in planning components is because the cooldown is really on the adaptation action and the planning component is what produce the plan for the action. The metrics that is computed by analyzing component might be useful in future analysis so we want the analyzing component to compute metrics and store it in the database for every monitoring periods but the actual plan is produced every 5 seconds.

The last part of the MAPE-K loop is the **executing components** which in our case just add or remove counters or switching scheduling policies. There is really no logic here. We just realize whatever plan the planning component gives.

Lastly, we want a dashboard so user can easily interact with our project to simulate different workloads. We allow users to specify their own initial counters and maximum number of counters, the number of concurrent users, whether they want adaptation and the duration of the simulation. On top of specifying these simulation parameters, the dashboard also gives real time metrics such as number of counters and wait times so the user have an understanding of the state of the current queue system.

Fig 2. shows the overall architecture of different components in our project and how they connect together to solve the problem. First, JMeter sends jobs to our queue system which is the managed system to simulate the process of jobs coming in with a random processing time. The queue system just act as a normal queue system that put jobs in a queue and completes these jobs with multiple counters. Then we have the MAPE-K loop where the monitor collects metrics from the queue system and send it to the analyzing components. The analyzing components computes metrics and

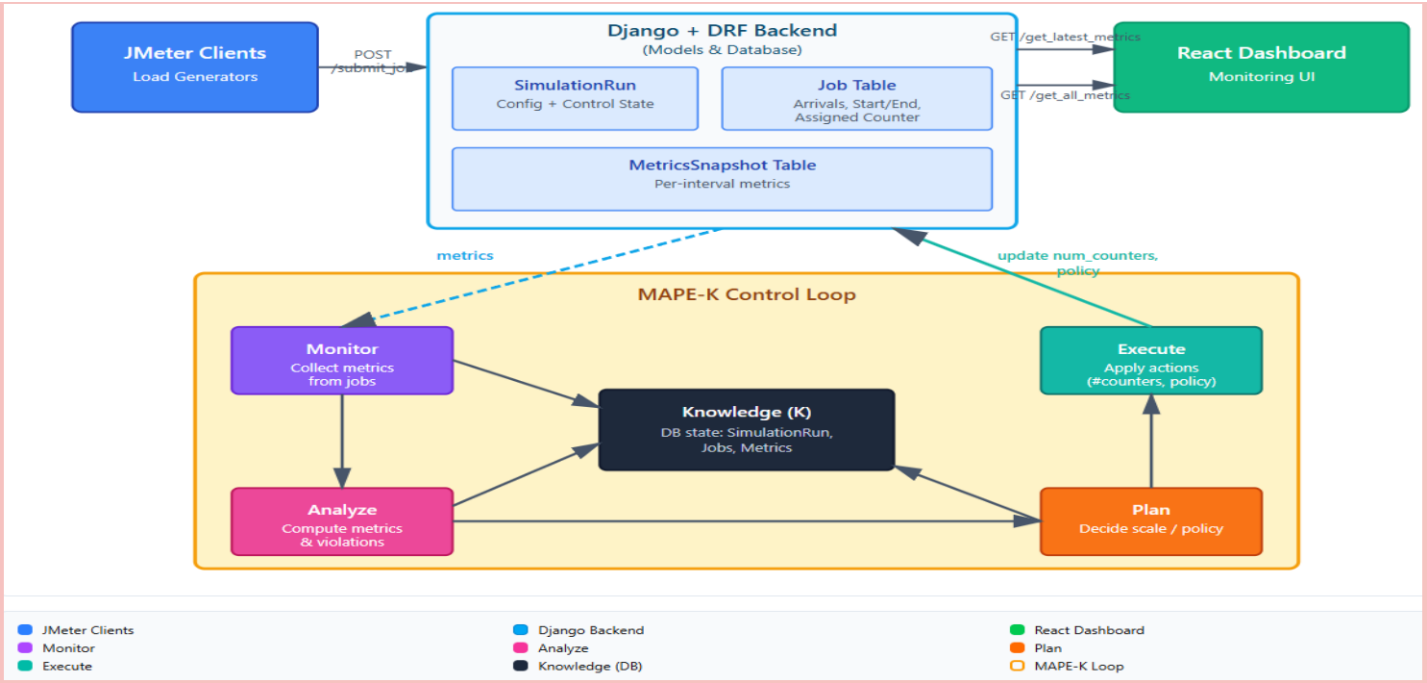

Fig. 2. Overall architecture of our project

send these metrics to planning component to generate a plan. The planning component figure out a plan such as add a counter or switch scheduling policy and send this plan to executing component. Lastly, the executing component realizes the plan to the queue system and our frontend dashboard will reflect the change and some metrics to the user. The knowledge is the center of the MAPE-K loop that contains our adaptation goals and metrics in the past.

III. Obtained Results

To talk about the implementation of our project, the first part of our project that we implemented is the backend simulation code where we simulate the environment of processing jobs in a queue with some number of counters. This part is implemented with django and python where we have a single queue that we can specify the queueing method the queue uses. We also initialize a fixed number of different counters to start with and we allow user to specify a maximum number of counters that they allow to use. These simulation elements are designed using views in django and to determine the next jobs to process when a counter becomes available, we basically just filter the set of all jobs that is waiting to be processed. To simulate the unpredictable traffic intensity and processing time, we ask JMeter to generate these for us where we really have no control over how frequent jobs are coming in and what exactly the processing time of each job will be. However, we do control one thing which is the maximum and minimum processing time of each job where no jobs exceeds the maximum processing time or drop below the minimum processing time. We add this restriction because we want to keep the simulation time short which is more efficient on testing purposes instead of running 30 minutes simulation to see a failure or mistake in our implementation. We restrict the processing time of each jobs to be between 1 to 50 ms so they are quick simulation that we can run to see whether our adaptation is effective or not but in reality, this processing time does not have to be restricted but the randomness in the processing time is still there in our simulation. We also allow the user to initialize the number of concurrent user they want and the duration of the simulation. After finishing the implementation of our django backend, we continue to develop the actual core of our self-adaptation system which is the MAPE-K Loop and it is broken into four parts which are Monitoring, Analyzing, Planning and Executing [3].

In the monitoring components, we basically just query the database for the information we needed such as average wait time, queue length and number of jobs currently in counters. We simply use the filter function call to query for these information and in the analyzing components, we use these information from the monitoring component to calculate the utilization and heavy-tail indicator. We implemented our own percentile function to help with calculating the heavy-tail indicator metrics. These two metrics that is calculated by the analyzing component is combined together with other information from the monitoring component into an object where we call it MetricsSnapshot and sent to the planning components. This object basically represents a snapshot of the current state of the queue system. Then the planning component extracts the different metrics and use simple if loop to generate a plan to execute. The 5 seconds cooldown

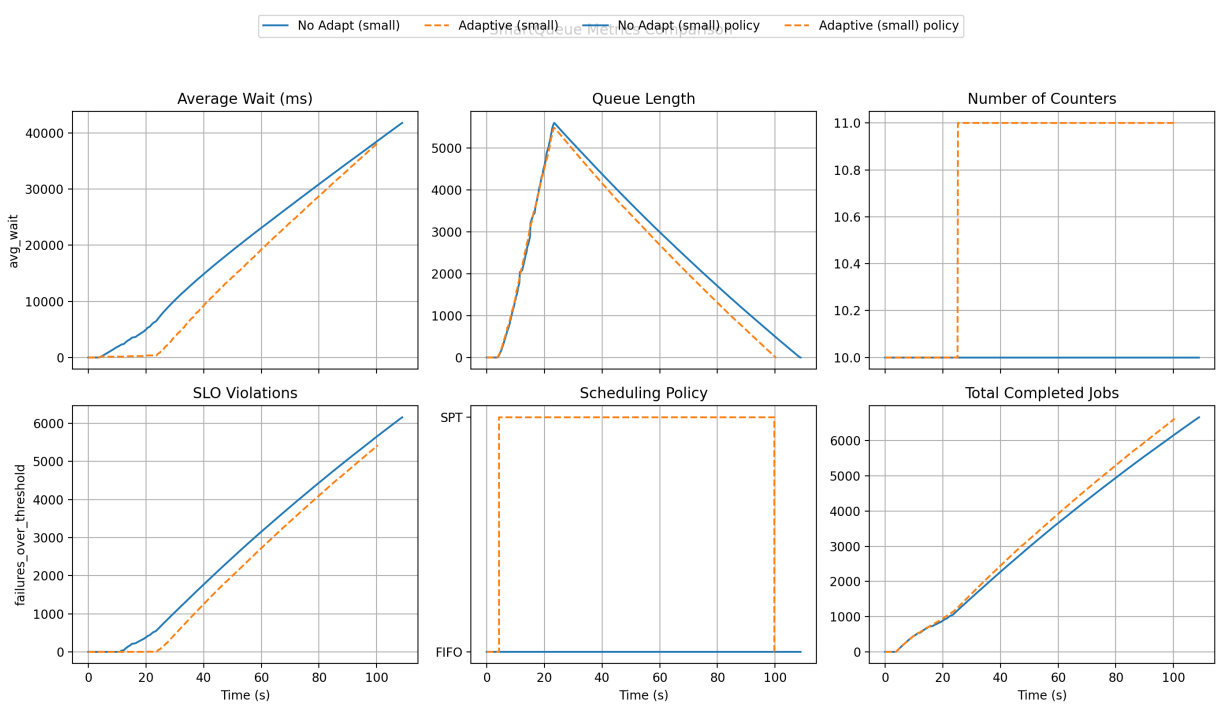

Fig. 3. Results of Small Workload

is enforced by storing the last adaptation time and if the last adaptation time is not more than 5 seconds from now, we give no adaptation needed plan to the executing components. Then this plan is sent to the executing component where it changes some parameters related to the current simulation such as the number of counters or the scheduling policies used for the current simulation. All MAPE-K loop are implemented with python and django.

Lastly, we implemented the frontend dashboard using react where we constructed some input text field that allows user to enter their choice for the simulation that they want to run (such as initial counter and duration). Then the react dashboard sends this to the django backend to start the simulation, and it will continuously call the API in the backend to extract metrics about the current simulation to graph these metrics in real time so the user can actually observe how different metrics change. Refer to **Fig 2** for visualization of the architecture.

Now, we will discuss the results of our evaluation. Because we want to evaluate under different circumstances and workload, we designed three different workloads to observe how queue system with adaptation performs compare to static configuration.

First, we have small workload where we choose the parameter for setup in **TABLE III**.

We can see that we keep the maximum number of counters only 10% more than the initial number of counters to stick with our resource constraint and if we look at **Fig 3** for output of small workload, we can see that the difference between static and adaptive queue system is not really big. The average wait time for adaptive system is always below the static configuration where the wait time of adaptive system starts increase later and at a slower speed compare to static system. If we look at queue length, static and adaptive system have roughly the same queue length when JMeter sends requests but after JMeter finishes sending requests, adaptive system finishes jobs in the queue faster than static system. Next, if we look at the number of failed jobs, we can see that adaptive system also performs better than static system by having about 1000 violation less compared to static system. Lastly, if we look at the total completed jobs, adaptive system takes less time to complete the same number of jobs. Overall, in small workload, the adaptive system that our project propose did not achieve the 30% reduce in violations (30% of about 6000 is 1800 but we reduce by only 1000 violations) but we can still observe the benefits of having adaptive systems and we believe that the affect of adaptation is not clearly shown under small workload because the system is not too overwhelmed with jobs so static system can still handle the workload.

TABLE III
SMALL WORKLOAD PARAMETER

Parameter	Value
concurrent users	10
initial counter	10
max counters	11
duration (s)	20

Then we did the same evaluation under medium workload that we define with the parameters in **TABLE IV**

Still, we keep the same behavior where the maximum number of counters is 10% more than initial counters. If we

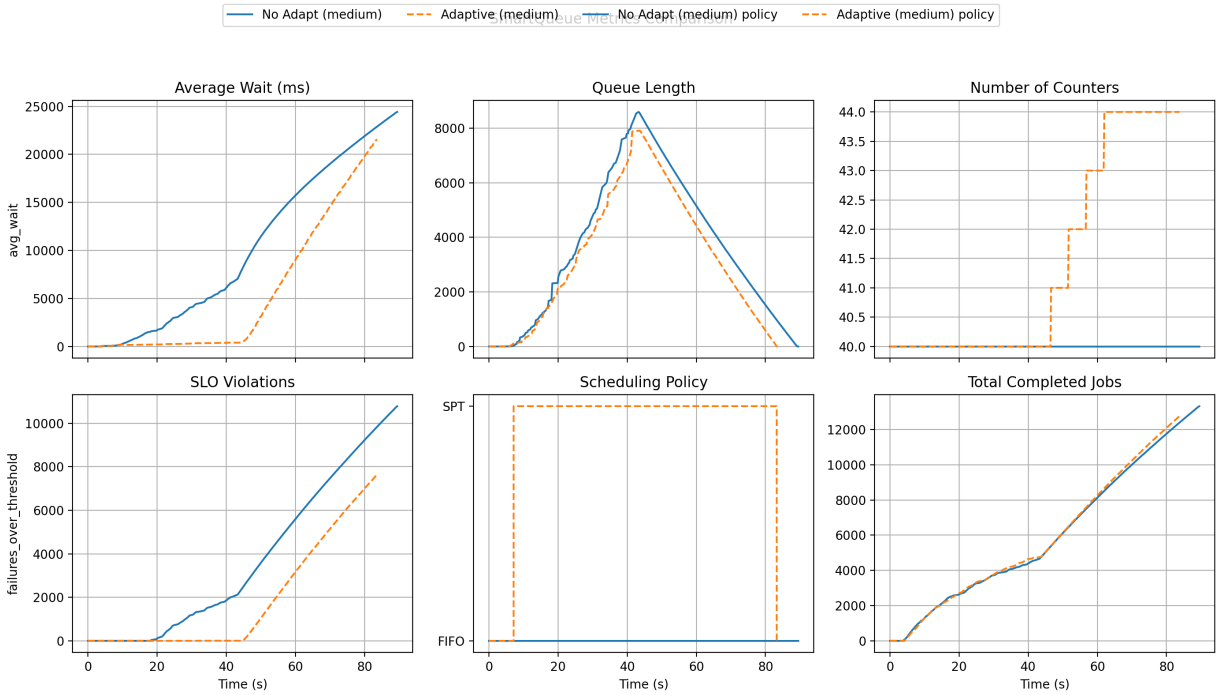

Fig. 4. Results of Medium Workload

TABLE IV
MEDIUM WORKLOAD PARAMETER

Parameter	Value
concurrent users	30
initial counter	40
max counters	44
duration (s)	40

look at **Fig 4** for the results, we see that the average wait time of adaptive system is again constantly below static system. The average wait time start to increase for adaptive system at about half wait through the simulation where the average wait time of static system start to increase right after we start the simulation. For the queue length, we also observe the same pattern where the queue length for adaptive system is constantly smaller than static system where the difference is more visible when JMeter stops sending requests and the two system starts to finish up the leftover jobs in the queue. This means that the adaptive function is faster in processing jobs compared to static configuration. If we look at the violation number, the static system has over 10,000 violations whereas the adaptive system has about 7,000 violation. This means that the adaptation action successfully reduce the violation number by 3000 which is about 30% of 10,000 so under medium workload, we achieved our quality requirement where we aim to reduce 30% using no more than 10% more counters. Lastly, if we look at the total job completed, the job numbers i roughly the same among the two system meaning that our adaptive system have this better performance not because of having more resources so it can process jobs faster, but instead by picking better jobs to process.

Last step of our evaluation is under large workload where we define it with the parameter in **TABLE V**

TABLE V
LARGE WORKLOAD PARAMETER

Parameter	Value
concurrent users	60
initial counter	100
max counters	110
duration (s)	80

If we look at the output in **Fig 5**, we observe that the average wait time still follow similar pattern as medium and small workload where adaptive system still remains below static system constantly. The average wait time increase for adaptive at about half way through the simulation where static system has a much early increase. Also, the increase for adaptive system is linear but for static system, it appears to be non-linear which increase faster so we believe that as we increase workload, the performance of adaptive system should be better than static system. The queue length between the two system is still roughly the same where as we process leftover jobs in the queue, adaptive system is slightly better than static system. If we look at number of failed jobs, we see that static system has almost 25,000 failed jobs where adaptive system has about 18,000 violations. The reduction in number of violation is approximately 30% with the 10% more counters used by adaptive system. For the total completed jobs, the adaptive system constantly completes more jobs than static system.

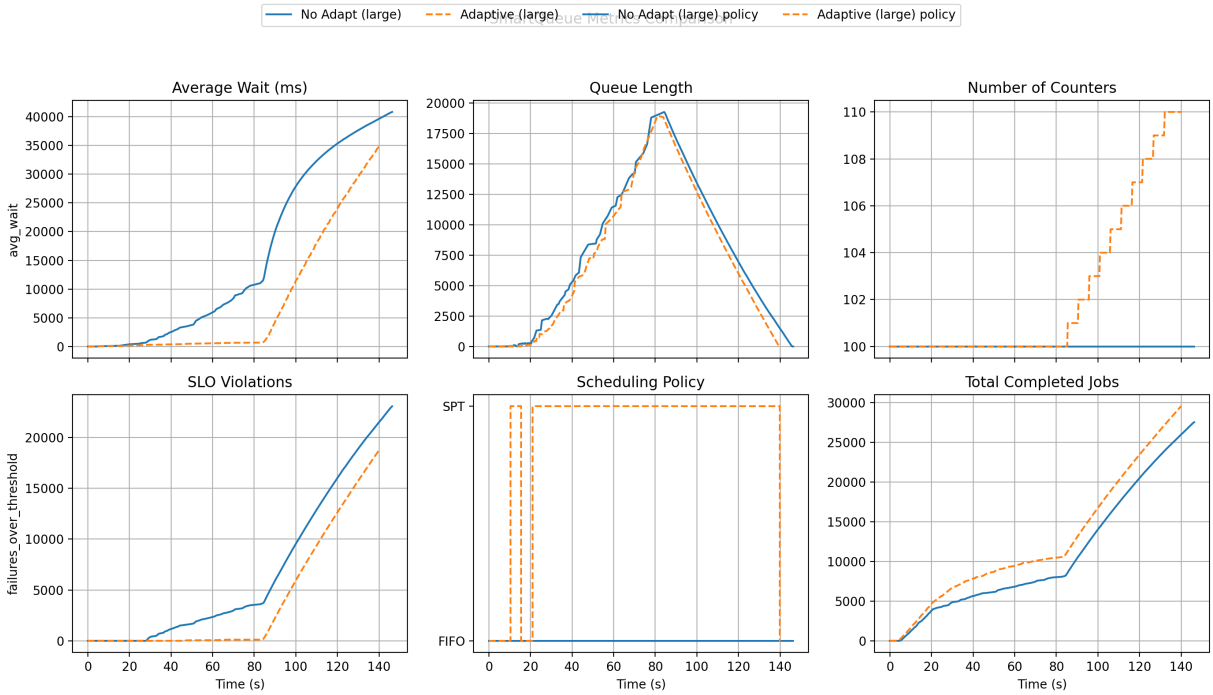

Fig. 5. Results of Large Workload

Overall, we believe that in all three workloads, the adaptive queue system performances better than static queue system and at larger workload, the difference in performance is more. Also, this is just for testing purposes and in real world, companies don't really have the 10% restriction. When the demand is small, 10 counters might solve all jobs but when the demand is large, companies might have 100 or more counters that can be used so in real world, the situation is different compared to simulation and companies will allow more than 10% more counters if quality of service can be maintained.

IV. Conclusion

To sum up, the problem and challenge that we try to solve in this project is the fact that static queueing system with fixed number of counters and scheduling policies relies on predicting the traffic intensity and arrival rate but in reality, traffic and workload is not predictable so it experiences delays and its quality of service also decreases. So instead of predicting traffic intensity and processing time, we make the queueing system adaptive where it can adjust its own configuration whenever environment changes.

Our solution periodically monitors metrics related to the queue system such as wait times and counters in use and whenever it detects something wrong like counters are overloaded or underloaded, our MAPE-K loop will come up with a plan such as increase or decrease counters or switching scheduling policies to adapt to this new environment. After the adaptation, the new configuration of the queue system should be in a better position to handle works in this new environment.

Based on the outcome of our evaluation, we observed that this approach results in better performance when we have a medium to large workload and it still results in a better performance under small workload but just not as significant as medium and large workload. Under medium and large workload, we manage to achieve a reduction of 30% in number of failed jobs with only at most 10% of increase in the number of counters compared to static queueing system. Furthermore, in the real world, we don't have this 10% restriction on resource because we have a fixed number for the maximum counter that is not necessarily at most 10% more than the initial counter. So the initial counter could start really small and as jobs come in, our solution will increase the number of counters accordingly without the 10% limit which could result in even a higher performance increase. Another circumstances that we did not evaluate on is the fact that our solution should reduce resource use when we have really small workloads where most counters are kept not used. Under this circumstances, we expect our solution to decrease the number of counters which saves resource used compare to static configuration where most counters are not used. We did not evaluate this case because first we are limited by the time of the course and second, JMeter seems to be crazy on sending jobs for the duration so it is hard to construct such as scenario and we don't have time to look into the solution for this.

Lastly, there is always room for improvement where there are areas that we can improve and limitations of our solution. One area that we can improve on is that two scheduling policies may not be enough for covering unpredictable arrival rate and processing time so we could incorporate more advanced scheduling policies to increase the performance of our system in some extreme cases. But we need to evaluate if the increase in performance is worth the effort of adding the scheduling policies. If we only get like 1% of increase in performance by adding a scheduling policies, it may not be a good idea to add it. Another idea that we can add to this project is incorporate AI and ML into the project. This project still uses static rules to determine when to make adaptation and these static rules can be replaced with AI and ML to make the solution more smart.

There are also limitations to our solution where if we switch to shortest processing times first, long jobs may be stuck in the queue all the way until the end which is really not fair in real world if the scenario is checking out at a shop so this is a potential limitation of the solution where the fairness of jobs is not considered. Also, in the context of a shop that may have VIP customers, there might be a priority associated with each jobs but our solution does not consider this priority so this is also a potential limitation to our solution.

V. Acknowledgment

We want to express our sincere gratitude to Professor Ladan for her excellent and insightful lectures and the teaching team of ECE750 for their valuable feedback on the project. We also want to gratefully acknowledge IBM for providing resources that we used to build up our knowledge in completing this project. Lastly, we want to acknowledge the use of multiple open-source tools in developing this project including python Django and Django REST framework for building backend, JMeter for generating workload, SQLite for data storage and React for frontend development.

References

[1] Harchol-Balter, M. (2014). Performance modeling and design of computer systems: Queueing theory in action. Cambridge University.

[2] Jain, R. (1991). The Art of Computer Systems Performance Analysis: Techniques for experimental design, Measurement, simulation, and modeling. Wiley.

[3] Malburg, L., Hoffmann, M., & Bergmann, R. (2023). Applying MAPE-K control loops for adaptive workflow management in Smart Factories. Journal of Intelligent Information Systems, 61(1), 83–111. https://doi.org/10.1007/s10844-022-00766-w

[4] https://github.com/shilinzhang-cs/ECE750-Project